



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRACTICA N°1

INTRODUCCIÓN A OPENGL

NOMBRE COMPLETO: García Cruz Andrea

N° de Cuenta: 316172697

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 01

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 2/09/26

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Edición de Código

```
CGIHC13 (Ámbito global)
1  #include <stdio.h>
2  #include <string.h>
3  #include <glew.h>
4  #include <glfw3.h>
5  #include <time.h>
6  #include <stdlib.h>
7  //Dimensiones de la ventana
8  const int WIDTH = 800, HEIGHT = 800;
9  GLuint VAO, VBO, shader;
10 //CAMBIO DE COLOR: DECLARACION DE VARIABLES
11 float CR = 1.0f; //ROJO
12 float CV = 0.0f; //VERDE
13 float CA = 0.0f; //AZUL
14 double TA = 0.0; //TIEMPO ATRAS
15 //LENGUAJE DE SHADER (SOMBRAS) GLSL
16 //Vertex Shader
17 //recibir color, salida Vcolor
18 static const char* vShader = "\n\
```

Se añadieron nuevas librerías time.h para que los colores fueran aleatorios siempre
stdlib.h para el uso de rand y otra función que ayudara con la aleatoriedad

Solo nos quedamos con estas variables del programa anterior

```
39 void CrearLETRAS()
40 {
41     GLfloat vertices[] = {
42         //PARA LA LETRA A
43         // IZQU. DIAGONAL
44         -0.25f, -0.4f, 0, -0.20f, -0.4f, 0, -0.175f, 0.1f, 0,
45         -0.20f, -0.4f, 0, -0.15f, 0.1f, 0, -0.175f, 0.1f, 0,
46         // DER. DIAGONAL
47         -0.10f, -0.4f, 0, -0.05f, -0.4f, 0, -0.125f, 0.1f, 0,
48         -0.05f, -0.4f, 0, -0.075f, 0.1f, 0, -0.125f, 0.1f, 0,
49         // PARA EL CENTRO
50         -0.19f, -0.15f, 0, -0.11f, -0.15f, 0, -0.11f, -0.2f, 0,
51         -0.19f, -0.15f, 0, -0.11f, -0.2f, 0, -0.19f, -0.2f, 0,
52         //PARA ARRIBA
53
54         //PARA LA LETRA G
55         // BARRA SUPERIOR
56         0.00f, 0.04f, 0, 0.22f, 0.04f, 0, 0.22f, 0.10f, 0,
57         0.00f, 0.04f, 0, 0.22f, 0.10f, 0, 0.00f, 0.10f, 0,
58         // BARRA IZQ
59         0.00f, -0.40f, 0, 0.06f, -0.40f, 0, 0.06f, 0.10f, 0,
60         0.00f, -0.40f, 0, 0.06f, 0.10f, 0, 0.00f, 0.10f, 0,
61         // BARRA INF.
62         0.00f, -0.40f, 0, 0.22f, -0.40f, 0, 0.22f, -0.34f, 0,
63         0.00f, -0.40f, 0, 0.22f, -0.34f, 0, 0.00f, -0.34f, 0,
64         // CURVITA: barra vertical corta (abajo-derecha)
65         0.16f, -0.40f, 0, 0.22f, -0.40f, 0, 0.22f, -0.12f, 0,
66         0.16f, -0.40f, 0, 0.22f, -0.12f, 0, 0.16f, -0.12f, 0,
```

Se crea la nueva función para generar las letras, con sus coordenadas correspondientes de aparición

```

65     0.16f,-0.40f,0, 0.22f,-0.40f,0, 0.22f,-0.12f,0,
66     0.16f,-0.40f,0, 0.22f,-0.12f,0, 0.16f,-0.12f,0,
67     // CURVITA: barra horizontal que cierra hacia el centro
68     0.11f,-0.18f,0, 0.22f,-0.18f,0, 0.22f,-0.12f,0,
69     0.11f,-0.18f,0, 0.22f,-0.12f,0, 0.11f,-0.12f,0,
70     //PARA LA LETRA C
71     //BARRA SUPERIOR
72     0.28f, 0.04f,0, 0.46f, 0.04f,0, 0.46f, 0.10f,0,
73     0.28f, 0.04f,0, 0.46f, 0.10f,0, 0.28f, 0.10f,0,
74     //BARRA IZQ
75     0.28f,-0.40f,0, 0.34f,-0.40f,0, 0.34f, 0.10f,0,
76     0.28f,-0.40f,0, 0.34f, 0.10f,0, 0.28f, 0.10f,0,
77     //BARRA INFERIOR
78     0.28f,-0.40f,0, 0.46f,-0.40f,0, 0.46f,-0.34f,0,
79     0.28f,-0.40f,0, 0.46f,-0.34f,0, 0.28f,-0.34f,0,
80     };

```

```

172 //CREAR VENTANA
173 GLFWwindow *mainWindow = glfwCreateWindow(WIDTH, HEIGHT, "HOLA SOY AGC", NULL, NULL);
174
175 if (!mainWindow)
176 {
177     printf("Fallo en crearse la ventana con GLFW");
178     glfwTerminate();
179     return 1;
180 }
181 //Obtener tamaño de Buffer
182 int BufferWidth, BufferHeight;
183 glfwGetFramebufferSize(mainWindow, &BufferWidth, &BufferHeight);

```

Se cambia el encabezado a nuestro gusto

CGIHC13 (Ámbito global)

```

202
203 //Llamada a las funciones creadas antes del main
204 CrearLETRAS();
205 CompileShaders();
206 srand((unsigned int)time(NULL));
207 TA = glfwGetTime();//TIEMPO DE INICIO PARA TRAPECIO Y ROMBO
208
209
210 //Loop mientras no se cierra la ventana
211 while (!glfwWindowShouldClose(mainWindow))
212 {
213     //Recibir eventos del usuario
214     glfwPollEvents();
215     //PROGRAMACIÓN PARA EL CAMBIO DE COLORES
216     double TAH= glfwGetTime();//Tiempo de ahora
217     if (TAH - TA >= 2.0)
218     {
219         CR = (float)rand() / RAND_MAX;
220         CV = (float)rand() / RAND_MAX;
221         CA = (float)rand() / RAND_MAX;
222         TA = TAH;
223     }
224

```

Se ejecuta una sola vez, antes de entrar al while para que rand() cambie una única vez por ejecución para no perder aleatoriedad real entre colores consecutivos

Normalizando el estado a un float



Problemas presentados

Comenzó con que se veían desfasadas las letras o algún triángulo no se veía bien, pero con ayuda de Claude, mi asesor de confianza logré ver porque ocurría eso. Además de la colocación correcta de *srand((unsigned int)time(NULL))*; porque no se podía cambiar de forma aleatoria los colores repetían igual.

Conclusión

En esta práctica logré familiarizarme con los conceptos fundamentales para el uso de OpenGL. Por medio de este ejercicio comprendí el flujo básico de renderizado, el uso de *glDrawArrays* para indicar cuantos vértices se deben de ocupar en cada llamada solicitada.

Reforcé que es necesario saber bien cuantos vértices se van a ocupar ya que todo hay que descomponerlo en triángulos, recordé el uso de la librería *stdlib.h* para la generación de números pseudoaleatorios mediante el uso de *rand()*, entendiendo así la función de *time.h*, para que inicie la secuencia *srand*, y así evitar repetir los mismos valores de color en cada ejecución. Asimismo, desarrollé más mi capacidad de depuración al identificar errores lógicos que no generaban fallas de compilación, pero sí afectaban directamente el resultado visual del programa. En general, esta práctica sentó las bases para trabajar con geometría más compleja en OpenGL.

Bibliografía en formato APA

Anthropic. (2026). Claude (Fable 5)[Large language model]. <https://claude.ai/>